Übungsblatt 8: I/O & Systemarchitektur

Mikrocomputertechnik 1

Musterlösung

Zielsetzung

In diesem Übungsblatt brechen Sie die Isolation der CPU auf. Im ersten Teil (Ripes) steuern Sie über Memory-Mapped I/O (MMIO) direkt Hardware-Register wie Schalter und LEDs an. Im zweiten Teil (Venus) nutzen Sie System Calls, um das Betriebssystem als Pförtner für Konsolenausgaben (Drucken von Text und Zahlen) um Hilfe zu bitten.

Aufgabe 1: Hardware-Kommunikation und Traps

Beantworten Sie die folgenden Fragen zu den Systemgrundlagen der Vorlesung.

MMIO und das `volatile`-Problem

In der C-Programmierung muss bei Pointern auf Memory-Mapped-I/O-(MMIO)-Adressen zwingend das Schlüsselwort `volatile` genutzt werden. Erklären Sie, warum I/O-Register „lebendig“ sind und was passiert, wenn der Compiler eine Leseschleife ohne `volatile` optimiert.

Polling vs. Interrupts

Was ist der größte Nachteil von Polling (Busy Waiting) auf Systemebene? Warum vergleicht man Interrupts stattdessen mit einer Türklingel?

Das CSR-System

Bei einem Interrupt springt die CPU in den Trap-Handler. Welche Aufgabe übernehmen dabei die speziellen Schattenregister `mtvec` und `mepc`?

Ihre Lösung

(Notieren Sie Ihre Antworten hier.)

Musterlösung Aufgabe 1

MMIO und `volatile`:

- MMIO-Register sind flüchtig: Ihr Wert ändert sich durch externe physikalische Einflüsse (z. B. Tastendruck), ohne dass die CPU schreibt.
- Ohne `volatile` nimmt der Compiler an, der Wert bleibe gleich, und optimiert wiederholte Loads weg.
- Das Programm liest die Hardware dann nicht mehr aus und bemerkt den Knopfdruck nie.

Polling vs. Interrupts:

- Polling hält die CPU in einer Endlosschleife auf hoher Last, verbraucht Strom und blockiert Multitasking.
- Ein Interrupt weckt die CPU asynchron auf („Türklingel“), sobald Hardware Daten liefert; dazwischen kann die CPU andere Arbeit erledigen.

CSR-System:

- `mtvec` speichert die Basisadresse der Interrupt Service Routine (Trap-Handler), zu der die Hardware bei einem Trap springt.
- `mepc` speichert den PC des unterbrochenen User-Codes, damit die CPU am Ende mit `mret` an die exakte Stelle vor der Unterbrechung zurückkehren kann.

Aufgabe 2: Bare-Metal I/O in Ripes (Switches & LEDs)

Öffnen Sie Ripes (Single-Cycle Processor). Wechseln Sie in den Reiter **I/O** und fügen Sie über das Plus-Symbol **Switches** und eine **LED Matrix** hinzu. Lesen Sie die Basisadressen unten links ab (typischerweise Switches: `0xF0000000`, LEDs: `0xF0000020`).

Aufgabenstellung

Programmieren Sie eine Endlosschleife, die permanent die Switches ausliest.

1. Filtern Sie mit einer logischen Maske (`andi`) exakt Schalter Nummer 3 (Bit 3) heraus.
2. Wenn der Schalter gedrückt ist, schalten Sie die erste LED in der Matrix auf Rot (Farbwert `0x00FF0000`).
3. Wenn der Schalter nicht gedrückt ist, schalten Sie die LED ab (`0x00000000`).

Ihre Lösung

(Fügen Sie Ihren Assembler-Code hier ein.)

Musterlösung Aufgabe 2

```
.text
    # Basisadressen laden
    li s1, 0xF0000000   # MMIO: Switches
    li s2, 0xF0000020   # MMIO: LED Matrix

Loop:
    lw t0, 0(s1)        # Schalterzustand lesen

    # Bit 3 isolieren (Maske: binär 1000 = 8)
    andi t1, t0, 8

    beq t1, zero, LedOff

LedOn:
    li t2, 0x00FF0000   # Rot
    j UpdateLED

LedOff:
    li t2, 0x00000000   # Aus / Schwarz

UpdateLED:
    sw t2, 0(s2)        # Farbe in LED-MMIO schreiben
    j Loop              # Polling
```

Aufgabe 3: OS-Syscalls in Venus (Konsole & EVA)

Schließen Sie Ripes und öffnen Sie den Venus-Simulator. Wir verlassen Bare-Metal und rufen das Betriebssystem (`ecall`) an, um Text auf die Konsole zu drucken.

Achtung: Die Venus-ABI erwartet die Service-ID in `a0` und das Argument in `a1`.

Aufgabenstellung

1. Legen Sie im `.data`-Segment einen nullterminierten String an: `.string "Ergebnis: "` (oder `.asciiz`).
2. Rufen Sie Service 4 (Print String) auf. `a1` benötigt die Speicheradresse des Strings (`la`).
3. Laden Sie die Konstante 21 in ein temporäres Register und verdoppeln Sie sie per Schiebeoperation (`slli`) oder Addition.
4. Rufen Sie Service 1 (Print Integer) auf, um die berechnete Zahl auszudrucken.
5. Beenden Sie das Programm sauber mit Service 10 (Exit).

Ihre Lösung

(Fügen Sie Ihren Assembler-Code hier ein.)

Musterlösung Aufgabe 3

```
.data
msg: .string "Ergebnis: "

.text
.globl main
main:
    # --- 1. Text drucken ---
    li a0, 4            # Print String
    la a1, msg
    ecall

    # --- 2. Rechnen ---
    li t0, 21
    slli t0, t0, 1      # t0 = 42

    # --- 3. Zahl drucken ---
    li a0, 1            # Print Integer
    mv a1, t0           # Argument in a1
    ecall

    # --- 4. Exit ---
    li a0, 10
    ecall
```

Aufgabe 4: Die Clobbering-Gefahr bei `ecall`

Ein Kommilitone hat folgenden Venus-Code geschrieben, um dreimal das Wort „Hallo“ zu drucken. Das Programm druckt „Hallo“ jedoch endlos (oder verhält sich unzuverlässig) und beendet sich nicht sauber.

```
.data
text: .string "Hallo\n"

.text
    li t0, 3            # Schleifenzähler
Loop:
    beq t0, zero, End

    li a0, 4            # Print String
    la a1, text
    ecall
```

```
    addi t0, t0, -1     # Zähler dekrementieren
    j Loop

End:
    li a0, 10
    ecall
```

Aufgabenstellung

1. Finden Sie den Fehler: Warum ist der Schleifenzähler in `t0` laut ABI nach dem `ecall` nicht garantiert — auch wenn Venus zufällig funktioniert?
2. Wie können Sie den Fehler am einfachsten beheben, ohne den Stack zu nutzen?

Ihre Lösung

(Notieren Sie Ihre Analyse hier.)

Musterlösung Aufgabe 4

Fehlerursache:

- Ein `ecall` ist ein ABI-Funktionsaufruf in den Kernel.
- Laut ABI darf das Betriebssystem Caller-Saved-Register (`t0`–`t6`, `a0`–`a7`) überschreiben (Clobbering).
- Venus erhält `t`-Register faktisch oft — das ist nicht garantiert und nicht portabel. Verlassen Sie sich nicht darauf.

Lösung ohne Stack:

- Nutzen Sie ein Callee-Saved-Register (z. B. `s0`).
- Mit `li s0, 3` und `addi s0, s0, -1` bleibt der Zähler nach dem `ecall` laut ABI erhalten (OS muss Callee-Saved bewahren).